Commitment Schemes

October 29, 2026

Blockstream Research

research.blockstream.com

github.com/BlockstreamResearch/zk-book



Plan

1. Commitments Overview
2. Hash-based Commitments
3. Pedersen Commitments
4. Vector Commitments
   - Pedersen Vector Commitment
   - Merkle Tree based Vector Commitment
5. Polynomial Commitments
   - Kate-Zaverucha-Goldberg (KZG)

Commitments Overview

Commitment Definition

Definition

A cryptographic commitment scheme allows one party to commit to a chosen statement without revealing the statement itself. The commitment can be revealed in full or in part at a later time, ensuring the integrity and secrecy of the original statement until the moment of disclosure.

Commitment Definition

Definition

A cryptographic commitment scheme allows one party to commit to a chosen statement without revealing the statement itself. The commitment can be revealed in full or in part at a later time, ensuring the integrity and secrecy of the original statement until the moment of disclosure.

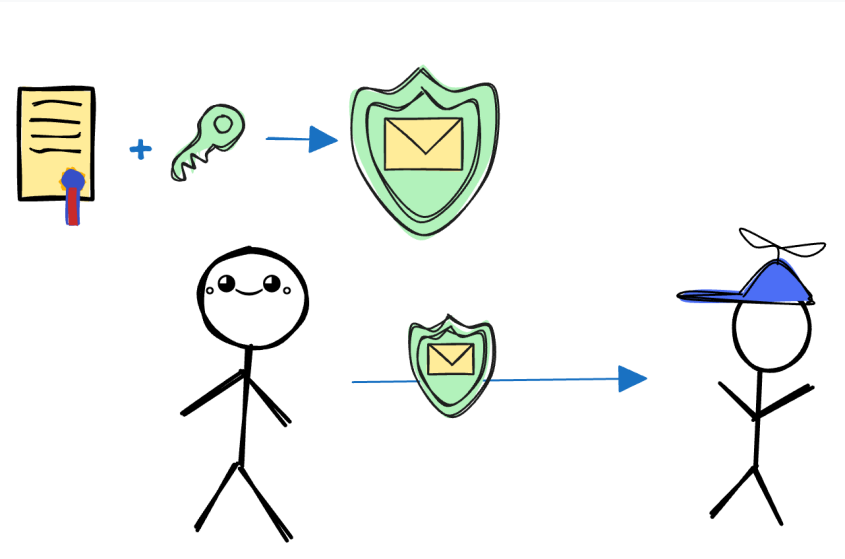


Figure: Overview of a commitment scheme

Commitment Definition

Definition

Commitment Scheme Π is a tuple of three algorithms:

$\Pi = (\text{Setup}, \text{Com}, \text{Open})$.

1. Setup(1^λ): returns public parameter pp for both committer and verifier;

Commitment Definition

Definition

Commitment Scheme Π is a tuple of three algorithms:
$\Pi = (\text{Setup}, \text{Com}, \text{Open})$.

1. $\text{Setup}(1^\lambda)$: returns public parameter pp for both committer and verifier;
2. $\text{Com}(\text{pp}, m)$: returns a commitment c to the message m using public parameters pp and, optionally, a secret opening hint r;

Commitment Definition

Definition

Commitment Scheme Π is a tuple of three algorithms:
$\Pi = (\text{Setup}, \text{Com}, \text{Open})$.

1. $\text{Setup}(1^\lambda)$: returns public parameter pp for both committer and verifier;
2. $\text{Com}(\text{pp}, m)$: returns a commitment c to the message m using public parameters pp and, optionally, a secret opening hint r;
3. $\text{Open}(\text{pp}, c, m, r)$: verifies the opening of the commitment c to the message m with an opening hint r.

Commitment Definition

Definition

Commitment Scheme Π is a tuple of three algorithms:
$\Pi = (\text{Setup}, \text{Com}, \text{Open})$.

1. $\text{Setup}(1^\lambda)$: returns public parameter pp for both committer and verifier;
2. $\text{Com}(\text{pp}, m)$: returns a commitment c to the message m using public parameters pp and, optionally, a secret opening hint r;
3. $\text{Open}(\text{pp}, c, m, r)$: verifies the opening of the commitment c to the message m with an opening hint r.

Remark

In many applications, the random component r is passed together with a message, which we write as $\text{Com}(m; r)$. Then, Open is trivial: it checks whether $\text{Com}(m; r) = c$.

Commitment Scheme Properties

Definition

1. *Hiding:* verifier should not learn any additional information about the message given only the commitment c.
   1.1. *Perfect hiding*: adversary with any computation capability tries even forever cannot understand what you have hidden.
   1.2. *Computationally hiding*: we assume that the adversary has limited computational resources and cannot try forever to recover hidden value.

Commitment Scheme Properties

Definition

1. *Hiding:* verifier should not learn any additional information about the message given only the commitment c.
   1.1. *Perfect hiding*: adversary with any computation capability tries even forever cannot understand what you have hidden.
   1.2. *Computationally hiding*: we assume that the adversary has limited computational resources and cannot try forever to recover hidden value.
2. *Binding:* prover could not find another message m_1 and open the commitment c without revealing the committed message m.
   2.1. *Perfect binding*: adversary with any computation capability tries even forever cannot find another m_1 that would result to the same c.
   2.2. *Computationally binding*: we assume that the adversary has limited computational resources and cannot try forever.

Hiding and Binding, Formally

Definition

Π is **hiding** if for any efficient adversary $\mathcal{A}$:

$$\left| \Pr\left[b = \widehat{b} \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda),\ m_0, m_1 \leftarrow \mathcal{A}(\mathsf{pp}) \\ b \leftarrow\!\!\$\ \{0,1\},\ c \leftarrow \mathsf{Com}(\mathsf{pp}, m_b),\ \widehat{b} \leftarrow \mathcal{A}(c) \end{array} \right] - \frac{1}{2} \right| = \mathsf{negl}(\lambda).$$

Hiding and Binding, Formally

Definition

Π is **hiding** if for any efficient adversary $\mathcal{A}$:

$$\left| \Pr\left[b = \widehat{b} \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda),\ m_0, m_1 \leftarrow \mathcal{A}(\mathsf{pp}) \\ b \leftarrow\!\$\ \{0,1\},\ c \leftarrow \mathsf{Com}(\mathsf{pp}, m_b),\ \widehat{b} \leftarrow \mathcal{A}(c) \end{array} \right] - \frac{1}{2} \right| = \mathsf{negl}(\lambda).$$

Definition

Π is **binding** if for any efficient adversary $\mathcal{A}$:

$$\Pr\left[\begin{array}{l} \mathsf{Com}(m_0; r_0) = \mathsf{Com}(m_1; r_1) \\ \text{and } m_0 \neq m_1 \end{array} \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda) \\ (m_0, m_1, r_0, r_1) \leftarrow \mathcal{A}(\mathsf{pp}) \end{array} \right] = \mathsf{negl}(\lambda).$$

Hiding and Binding, Formally

Definition

Π is **hiding** if for any efficient adversary $\mathcal{A}$:

$$\left| \Pr\left[b = \widehat{b} \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda),\ m_0, m_1 \leftarrow \mathcal{A}(\mathsf{pp}) \\ b \leftarrow\!\$\ \{0,1\},\ c \leftarrow \mathsf{Com}(\mathsf{pp}, m_b),\ \widehat{b} \leftarrow \mathcal{A}(c) \end{array} \right] - \frac{1}{2} \right| = \mathsf{negl}(\lambda).$$

Definition

Π is **binding** if for any efficient adversary $\mathcal{A}$:

$$\Pr\left[\begin{array}{l} \mathsf{Com}(m_0; r_0) = \mathsf{Com}(m_1; r_1) \\ \text{and } m_0 \neq m_1 \end{array} \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda) \\ (m_0, m_1, r_0, r_1) \leftarrow \mathcal{A}(\mathsf{pp}) \end{array} \right] = \mathsf{negl}(\lambda).$$

Remark

The scheme is **perfectly** hiding/binding if the left-hand sides above are exactly zero for any (not necessarily efficient) adversary. Perfect hiding and perfect binding cannot be achieved simultaneously.

Hash-based Commitments

Hash-based Commitments

As the name implies, we are using a cryptographic hash function H in such scheme.

Definition

1: Prover selects a message m from a message space $\mathcal{M}$ which he wants to commit to: $m \in \mathcal{M}$.

Hash-based Commitments

As the name implies, we are using a cryptographic hash function H in such scheme.

Definition

1: Prover selects a message m from a message space $\mathcal{M}$ which he wants to commit to: $m \in \mathcal{M}$.

2: Prover samples random value r (usually called blinding factor) from a challenge space $\mathcal{C} \subset \mathbb{Z}$: $r \leftarrow\!\!\$\ \mathcal{C}$.

Hash-based Commitments

As the name implies, we are using a cryptographic hash function H in such scheme.

Definition

1: Prover selects a message m from a message space $\mathcal{M}$ which he wants to commit to: $m \in \mathcal{M}$.

2: Prover samples random value r (usually called blinding factor) from a challenge space $C \subset \mathbb{Z}$: $r \leftarrow\!\!\$\ C$.

3: Both values will be concatenated and hashed with the hash function H to produce the commitment: $c \leftarrow \mathrm{H}(m \parallel r)$.

Hash-based Commitments

As the name implies, we are using a cryptographic hash function H in such scheme.

Definition

1: Prover selects a message m from a message space $\mathcal{M}$ which he wants to commit to: $m \in \mathcal{M}$.

2: Prover samples random value r (usually called blinding factor) from a challenge space $\mathcal{C} \subset \mathbb{Z}$: $r \leftarrow\!\!\$\ \mathcal{C}$.

3: Both values will be concatenated and hashed with the hash function H to produce the commitment: $c \leftarrow \mathrm{H}(m \parallel r)$.

Lemma

Given a collision-resistant hash function H, the hash-based commitment is computationally binding. Moreover, it is hiding in the random oracle model for H.

Pedersen Commitments

Pedersen Commitment

Pedersen commitments allow us to represent arbitrarily large vectors with a single elliptic curve point. Pedersen commitment uses a public group $\mathbb{G}$ of order q (written multiplicatively) and two random public generators g and h: $h = g^{\rho}$. Secret parameter ρ should be unknown to anyone, otherwise the *binding* property of the commitment scheme will be violated.

Pedersen Commitment

Pedersen commitments allow us to represent arbitrarily large vectors with a single elliptic curve point. Pedersen commitment uses a public group $\mathbb{G}$ of order q (written multiplicatively) and two random public generators g and h: $h = g^{\rho}$. Secret parameter ρ should be unknown to anyone, otherwise the *binding* property of the commitment scheme will be violated.

Note: Transparent random points generation

User can pick the publicly known number (like x coordinate of group generator g), calculate $x_i = \mathrm{H}(x \parallel i)$ and corresponding y_i. Check whether (x_i, y_i) is in the elliptic curve group. Repeat the process for sequential $i = 1, 2, \ldots$ until point (x_i, y_i) is in the elliptic curve group.

Pedersen Commitment

Definition

Pedersen commitment scheme algorithm:

1: Prover and Verifier agree on points g and h in an elliptic curve point group $\mathbb{G}$, $q := \#\mathbb{G}$ is the order of the group.

Pedersen Commitment

Definition

Pedersen commitment scheme algorithm:

1: Prover and Verifier agree on points g and h in an elliptic curve point group $\mathbb{G}$, $q := \#\mathbb{G}$ is the order of the group.

2: Prover selects a value $m \in \mathbb{Z}_q$ to commit and a blinder factor $r \leftarrow\!\$\ \mathbb{Z}_q$.

Pedersen Commitment

Definition

Pedersen commitment scheme algorithm:

1: Prover and Verifier agree on points g and h in an elliptic curve point group $\mathbb{G}$, $q := \#\mathbb{G}$ is the order of the group.

2: Prover selects a value $m \in \mathbb{Z}_q$ to commit and a blinder factor $r \leftarrow\!\!\$\ \mathbb{Z}_q$.

3: Prover generates a commitment and sends it to the Verifier: $c \leftarrow g^m h^r$.

Pedersen Commitment

Definition

Pedersen commitment scheme algorithm:

1: Prover and Verifier agree on points g and h in an elliptic curve point group $\mathbb{G}$, $q := \#\mathbb{G}$ is the order of the group.

2: Prover selects a value $m \in \mathbb{Z}_q$ to commit and a blinder factor $r \leftarrow\$ \mathbb{Z}_q$.

3: Prover generates a commitment and sends it to the Verifier: $c \leftarrow g^m h^r$.

During the opening stage, prover reveals (m, r) to the verifier.

To check the commitment, verifier computes: $c' \leftarrow g^m h^r$.

If $c' = c$, prover has revealed the correct pair (m, r).

Pedersen Commitment: Security

Lemma

The Pedersen commitment scheme is *perfectly* hiding and *computationally* binding under the discrete logarithm assumption.

Pedersen Commitment: Security

Lemma

The Pedersen commitment scheme is *perfectly* hiding and *computationally* binding under the discrete logarithm assumption.

In case the discrete logarithm ρ of h is leaked, the *binding* property can be violated by the *prover*:

$$c = g^m h^r = g^m g^{r\rho} = g^{m+r\rho}$$

Pedersen Commitment: Security

Lemma

The Pedersen commitment scheme is *perfectly* hiding and *computationally* binding under the discrete logarithm assumption.

In case the discrete logarithm ρ of h is leaked, the *binding* property can be violated by the *prover*:

$$c = g^m h^r = g^m g^{r\rho} = g^{m+r\rho}$$

For example, $(m + \rho, r - 1)$ will have the same commitment value:

$$g^{m+\rho+(r-1)\rho} = g^{m+\rho-\rho+r\rho} = g^{m+r\rho}$$

Pedersen Commitment Aggregation

Pedersen commitment has a significant advantage over hash-based commitments: the former are additively homomorphic and thus can be accumulated. Consider two pairs $(m_0, r_0), (m_1, r_1)$.

Pedersen Commitment Aggregation

Pedersen commitment has a significant advantage over hash-based commitments: the former are additively homomorphic and thus can be accumulated. Consider two pairs $(m_0, r_0), (m_1, r_1)$.

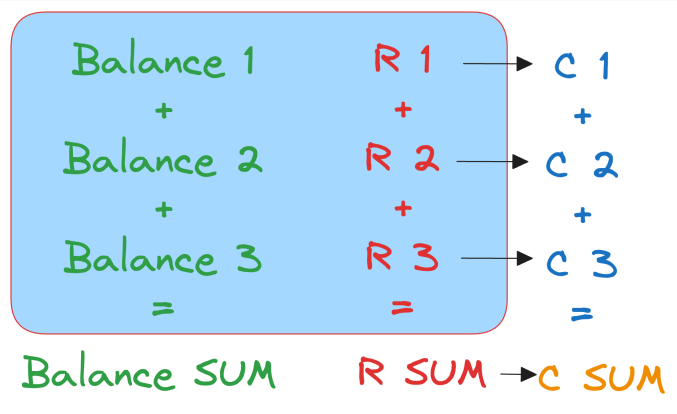
$$c_0 \leftarrow g^{m_0} h^{r_0}, \quad c_1 \leftarrow g^{m_1} h^{r_1}, \quad c_{\text{agg}} \leftarrow c_0 c_1 = g^{m_0+m_1} h^{r_0+r_1}.$$

Pedersen Commitment Aggregation

Pedersen commitment has a significant advantage over hash-based commitments: the former are additively homomorphic and thus can be accumulated. Consider two pairs $(m_0, r_0), (m_1, r_1)$.

$$c_0 \leftarrow g^{m_0} h^{r_0}, \quad c_1 \leftarrow g^{m_1} h^{r_1}, \quad c_{\text{agg}} \leftarrow c_0 c_1 = g^{m_0+m_1} h^{r_0+r_1}.$$

This works for any number of commitments, so we can encode as many points as we like in a single one.



Vector Commitments

Pedersen Vector Commitment

Suppose we have a set of random elliptic curve points $(g_0, \ldots, g_{n-1}, h)$ of cyclic group $\mathbb{G}$ (that nobody knows the discrete logarithm of), a vector $\mathbf{m} = (m_0, \ldots, m_{n-1}) \in \mathbb{Z}_q^n$ and a random value $r \leftarrow\!\!\$\ \mathbb{Z}_q$. We can do the following:

$$c \leftarrow h^r \prod_{i \in [n]} g_i^{m_i} = g_0^{m_0} g_1^{m_1} \cdots g_{n-1}^{m_{n-1}} h^r.$$

Pedersen Vector Commitment

Suppose we have a set of random elliptic curve points $(g_0, \ldots, g_{n-1}, h)$ of cyclic group $\mathbb{G}$ (that nobody knows the discrete logarithm of), a vector $\mathbf{m} = (m_0, \ldots, m_{n-1}) \in \mathbb{Z}_q^n$ and a random value $r \leftarrow\!\!\$\ \mathbb{Z}_q$. We can do the following:

$$c \leftarrow h^r \prod_{i \in [n]} g_i^{m_i} = g_0^{m_0} g_1^{m_1} \cdots g_{n-1}^{m_{n-1}} h^r.$$

Since the *Prover* does not know the discrete logarithm of the generators, he can only reveal $\mathbf{m}$ to produce c later: he cannot produce another vector.

Pedersen Vector Commitment

Suppose we have a set of random elliptic curve points $(g_0, \ldots, g_{n-1}, h)$ of cyclic group $\mathbb{G}$ (that nobody knows the discrete logarithm of), a vector $\mathbf{m} = (m_0, \ldots, m_{n-1}) \in \mathbb{Z}_q^n$ and a random value $r \leftarrow\$ \mathbb{Z}_q$. We can do the following:

$$c \leftarrow h^r \prod_{i \in [n]} g_i^{m_i} = g_0^{m_0} g_1^{m_1} \cdots g_{n-1}^{m_{n-1}} h^r.$$

Since the *Prover* does not know the discrete logarithm of the generators, he can only reveal $\mathbf{m}$ to produce c later: he cannot produce another vector.

Prover can later open the commitment by revealing the vector $\mathbf{m}$ and a blinding term r.

Merkle Tree Commitments

A naive approach for a vector commitment would be to hash the whole vector.

Merkle Tree Commitments

A naive approach for a vector commitment would be to hash the whole vector. More sophisticated scheme uses divide-and-conquer approach by building a binary tree out of vector elements.

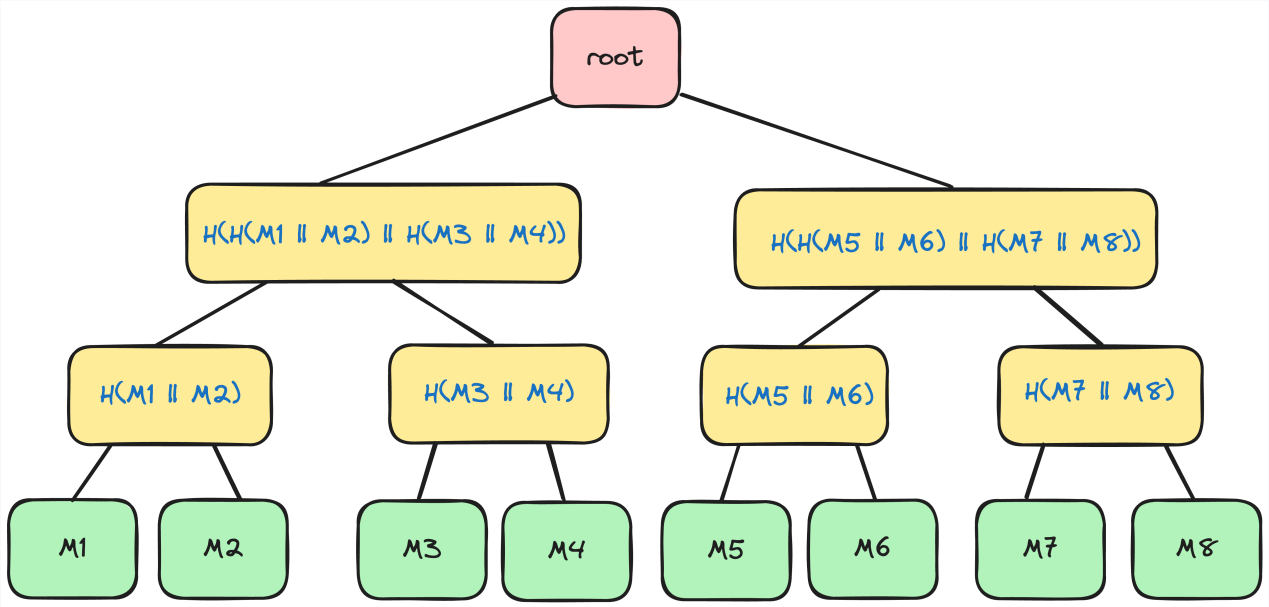


Figure: Merkle Tree structure

Merkle Tree Proof (MTP)

To prove the inclusion of element into the tree, a corresponding Merkle Branch is used. It allows to perform selective disclosure of the elements without revealing all of them at once.

Merkle Tree Proof (MTP)

To prove the inclusion of element into the tree, a corresponding Merkle Branch is used. It allows to perform selective disclosure of the elements without revealing all of them at once.

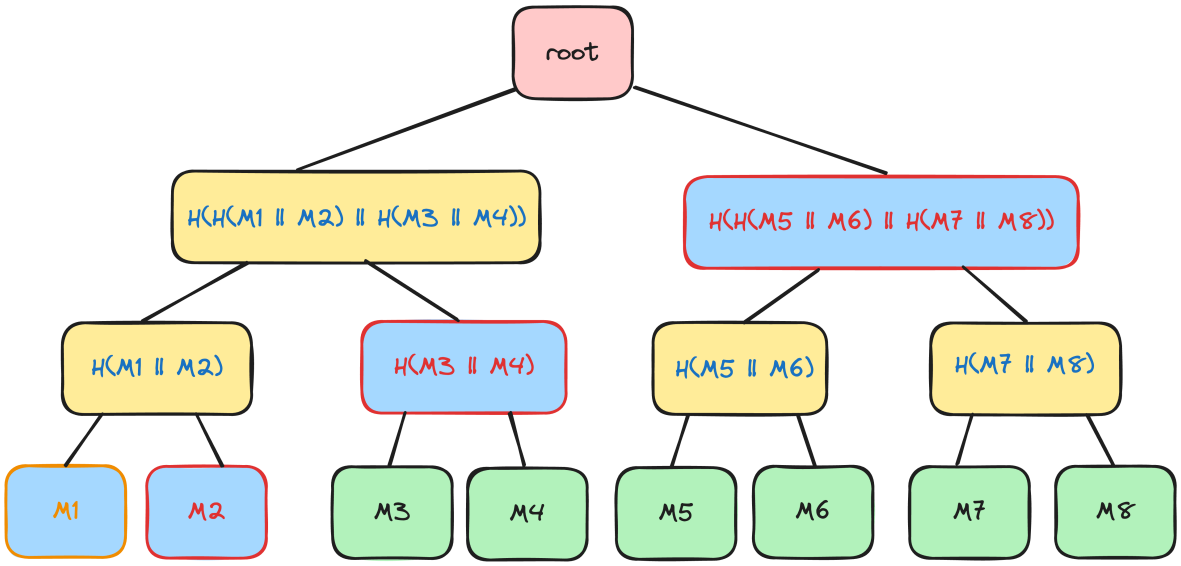


Figure: Merkle Tree inclusion proof branch for M_1: $(M_2, \mathrm{H}(M_3 \parallel M_4), \mathrm{H}(\mathrm{H}(M_5 \parallel M_6) \parallel \mathrm{H}(M_7 \parallel M_8)))$.

Polynomial Commitments

Polynomial Commitment

Definition

Polynomial commitment can be used to prove that the committed polynomial satisfies certain properties (passes through a certain point (ρ, y)), without revealing what the polynomial is. The commitment is generally succinct, which means that it is much smaller than the polynomial it represents.

Polynomial Commitment

Definition

Polynomial commitment can be used to prove that the committed polynomial satisfies certain properties (passes through a certain point (ρ, y)), without revealing what the polynomial is. The commitment is generally succinct, which means that it is much smaller than the polynomial it represents.

Remark

Typically, polynomial commitment scheme is a heart of any complex *zero-knowledge* proof system.

KZG Commitment. Simplified example

Given the polynomial: $f(T) = T^3 - 15T^2 + 71T - 103$

Prove that $f(3) = 2$

KZG Commitment. Simplified example

Given the polynomial: $f(T) = T^3 - 15T^2 + 71T - 103$

Prove that $f(3) = 2$

$f(3) = 2 \Longrightarrow 3$ is a root of polynomial $f(T) - 2$

KZG Commitment. Simplified example

Given the polynomial: $f(T) = T^3 - 15T^2 + 71T - 103$

Prove that $f(3) = 2$

$f(3) = 2 \implies 3$ is a root of polynomial $f(T) - 2$

Proof:

$$g(T) = \frac{f(T) - 2}{T - 3} = \frac{(T^3 - 15T^2 + 71T - 103) - 2}{T - 3} = T^2 - 12T + 35$$

KZG Commitment. Simplified example

Given the polynomial: $f(T) = T^3 - 15T^2 + 71T - 103$

Prove that $f(3) = 2$

$f(3) = 2 \implies 3$ is a root of polynomial $f(T) - 2$

Proof:

$$g(T) = \frac{f(T) - 2}{T - 3} = \frac{(T^3 - 15T^2 + 71T - 103) - 2}{T - 3} = T^2 - 12T + 35$$

Verify: $g(T) \cdot (T - 3) = f(T) - 2$

KZG Commitment

The KZG (Kate-Zaverucha-Goldberg) is a polynomial commitment scheme:

KZG Commitment

The KZG (Kate-Zaverucha-Goldberg) is a polynomial commitment scheme:

One-time "Powers-of-tau" trusted setup stage. During trusted setup a set of elliptic curve points is generated. Let g_1 be a generator of some pairing-friendly elliptic curve group $\mathbb{G}_1$ of order q, $\tau \leftarrow\$ \mathbb{Z}_q$ some random value and d be such that we want to commit to polynomials $f \in \mathbb{F}_q[T]^{<d}$.

$$\mathsf{pp} := \{g_1^{\tau^i}\}_{i\in[d]} = (g_1, g_1^{\tau}, g_1^{\tau^2}, \ldots, g_1^{\tau^{d-1}})$$

KZG Commitment

The KZG (Kate-Zaverucha-Goldberg) is a polynomial commitment scheme:

One-time "Powers-of-tau" trusted setup stage. During trusted setup a set of elliptic curve points is generated. Let g_1 be a generator of some pairing-friendly elliptic curve group $\mathbb{G}_1$ of order q, $\tau \leftarrow\$ \mathbb{Z}_q$ some random value and d be such that we want to commit to polynomials $f \in \mathbb{F}_q[T]^{<d}$.

$$\mathsf{pp} := \{g_1^{\tau^i}\}_{i\in[d]} = (g_1, g_1^{\tau}, g_1^{\tau^2}, \ldots, g_1^{\tau^{d-1}})$$

Toxic Waste

Parameter τ should be deleted after the ceremony. If it is revealed, the commitment scheme can be broken. This parameter is usually called the *toxic waste*.

KZG Commitment

Commit to polynomial. Given the polynomial $f(T) = \sum_{i \in [d]} f_i T^i$, compute the commitment $\mathsf{com}(f) \leftarrow g_1^{f(\tau)}$ using the trusted setup.

KZG Commitment

Commit to polynomial. Given the polynomial $f(T) = \sum_{i\in[d]} f_i T^i$, compute the commitment $\mathsf{com}(f) \leftarrow g_1^{f(\tau)}$ using the trusted setup.

Although the committer cannot compute $g_1^{f(\tau)}$ directly since the value of τ is unknown, he can compute it using values $\mathsf{pp} = (g_1, g_1^{\tau}, \ldots, g_1^{\tau^{d-1}})$:

$$g_1^{f(\tau)} = g_1^{\sum_{i\in[d]} f_i \tau^i} = \prod_{i\in[d]} g_1^{f_i \tau^i} = \prod_{i\in[d]} (g_1^{\tau^i})^{f_i}.$$

KZG Commitment

Prove an evaluation. Given evaluation $f(\rho) = y$, compute the proof $\text{com}(g) \leftarrow g_1^{g(\tau)}$, where

$$g(T) = \frac{f(T) - y}{T - \rho}.$$

KZG Commitment

Prove an evaluation. Given evaluation $f(\rho) = y$, compute the proof $\text{com}(g) \leftarrow g_1^{g(\tau)}$, where

$$g(T) = \frac{f(T) - y}{T - \rho}.$$

Polynomial g is called "quotient polynomial" and exists if and only if $f(\rho) = y$. The existence of this quotient polynomial serves as a proof of the evaluation.

KZG Commitment

Verify the proof. Given a commitment $\mathrm{com}(f) = g_1^{f(\tau)}$, an evaluation $f(\rho) = y$ and a proof $\mathrm{com}(g) = g_1^{g(\tau)}$, we need to ensure that $g(\tau) \cdot (\tau - \rho) = f(\tau) - y$.

KZG Commitment

Verify the proof. Given a commitment $\text{com}(f) = g_1^{f(\tau)}$, an evaluation $f(\rho) = y$ and a proof $\text{com}(g) = g_1^{g(\tau)}$, we need to ensure that $g(\tau) \cdot (\tau - \rho) = f(\tau) - y$.

This can be done without knowledge of τ using the pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ (the trusted setup additionally publishes $g_2^\tau \in \mathbb{G}_2$):

$$e(\text{com}(f)/g_1^y, g_2) = e(\text{com}(g), g_2^\tau / g_2^\rho)$$

KZG Commitment

Verify the proof. Given a commitment $\text{com}(f) = g_1^{f(\tau)}$, an evaluation $f(\rho) = y$ and a proof $\text{com}(g) = g_1^{g(\tau)}$, we need to ensure that $g(\tau) \cdot (\tau - \rho) = f(\tau) - y$.

This can be done without knowledge of τ using the pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ (the trusted setup additionally publishes $g_2^\tau \in \mathbb{G}_2$):

$$e(\text{com}(f)/g_1^y, g_2) = e(\text{com}(g), g_2^\tau / g_2^\rho)$$

Polynomial commitment schemes such as KZG are used in zero knowledge proof system to encode circuit constraints as a polynomial, so that verifier could check random points to ensure that the constraints are met.

Thank you for your attention



research.blockstream.com

github.com/BlockstreamResearch/zk-book



Blockstream